Exploratory Data Analysis: Understanding Your Data

> **What is this module for?**
> When you receive a raw dataset for a Machine Learning project, you should never feed it directly into an algorithm. This module covers the foundational stage of data exploration, showing you how to inspect structural shapes, catch hidden data issues, view statistical matrices, and extract initial structural patterns.

1. Data Dimensions & Volume

Before executing transformations, you must grasp the scale of the information you are handling. Knowing how many samples (rows) and attributes (columns) exist helps you evaluate computation requirements and baseline matrix structures.

```
import pandas as pd

df = pd.read_csv('titanic.csv')

# Returns a tuple representing dimensional shapes (rows, columns)
print("Data Matrix Layout:", df.shape)
```

2. Structural Preview & Bias Avoidance

While looking at the top rows of a dataset is common practice, it can often introduce unconscious bias if the raw records are sorted sequentially or grouped systematically. Utilizing random sampling gives you a far more honest look at the distributed variations across the entire file architecture.

```
# Standard top row preview
print(df.head())

# Recommended: Extract a random selection to eliminate ordered sequence bias
print(df.sample(5))
```

3. Data Types & Memory Auditing

Inspecting how attributes are represented in machine memory helps you verify if numeric values are stored cleanly or if categories are occupying bloated spaces. Auditing schema properties using summary overviews exposes structural details, counts, and footprints.

```
# Evaluates data types, counts of clean entries, and memory footprints
df.info()
```

4. Missing Value Assessment

Missing observations are highly destructive to machine learning models. You must always run an early scan across all columns to evaluate the volume of blanks so you can intelligently plan whether to impute values or drop incomplete attributes.

```
# Calculates total occurrences of blank data markers per column attribute
missing_counts = df.isnull().sum()
print(missing_counts)
```

5. Descriptive Mathematical Summary

Reviewing statistical distributions across your columns exposes central tendencies, spreads, and potential outliers. Generating high-level math matrices instantly summarizes the behavior of all your numeric variables.

```
# Outputs structural summaries: mean, deviations, minimums, percentiles, and ma
print(df.describe())
```

6. Duplicate Row Elimination

Duplicate entries can overfit your machine learning models and distort performance metrics. Checking for identical rows ensures your models are built on unique, verified historical patterns.

```
# Counts the total number of perfectly mirrored row entries across the matrix
print("Total Duplicate Rows:", df.duplicated().sum())
```

7. Pearson Feature Correlation

Features are rarely independent. Evaluating continuous linear relationships via Pearson's correlation coefficient shows how heavily columns influence each other, helping you identify redundant columns that do not contribute new value.

```
# Computes cross-correlation matrices between continuous variables (-1 to +1 ra
correlation_matrix = df.corr()
print(correlation_matrix['Survived'])
```

> **Key Developer Rule:** Always review feature correlation values. Identifying continuous relationships allows you to remove completely redundant or uninformative columns before training your algorithms, which optimizes both your memory footprint and model accuracy!